

POSSESSION OF MOBILES IN EXAM IS UFM PRACTICE.

Name UTKARSH GUPTA

Enrollment No. 2401020065

Jaypee Institute of Information Technology, Noida
Test-2 Examination, Odd 2024
B. Tech Semester: 1st

Course Title: Software Development Fundamentals-I
Course Code: 15B11CI111

Max. Time: 1 Hrs
Max. Marks: 20

Course Outcomes		Cognitive Levels
CO1	Explain the logic for solving problems considering various phases of Software Development Life Cycle and depicting them using algorithms and flowcharts	Understand (Level 2)
CO2	Explain basics of C programming concepts to make decision for solving problems	Understand (Level 2)
CO3	Demonstrate and contrast different methods for writing modular programs in C	Understand (Level 2)
CO4	Use various C programming constructs to implement iteration, and recursion	Apply (Level 3)
CO5	Apply and implement arrays, pointers structures and file handling for solving real-world problems	Apply (Level 3)

Note: Attempt all the Questions.

Q1. [CO3, (Understanding), 4 marks] Write a C program to evaluate a polynomial using functions. The program should begin by prompting the user to input the degree of the polynomial, denoted as 'n', which must be a non-negative integer. Subsequently, the user should enter the coefficients of the polynomial as an array. Implement a function named evaluatePolynomial that accepts the degree 'n', the array of coefficients, and a value 'x' as parameters to perform the evaluation. It is assumed that both the coefficients and 'x' value will be integers.

Example-Input: Enter the degree of the polynomial: 3

Enter the coefficients (from highest degree to lowest): 2 3 -4 5

Enter the value of x: 2

Output: $2x^3 + 3x^2 - 4x + 5 = 25$

Q2. [CO5, (Applying), 4 marks] Write a C program that identifies the contiguous sub array within a one-dimensional array of integers that has the largest sum. the program should prompt the user to input the size 'n' of the array, followed by integer elements of the array. The elements can be both positive and negative integers. Your program should determine the subarray that yields the maximum sum and print both its maximum sum and the corresponding subarray.

Example-Input: Enter value of n: 9

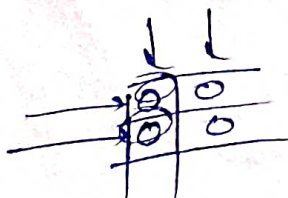
Enter elements of array = -2 1 -3 4 -1 2 1 -5 4

Output: Largest sum is 6

Subarray with largest sum is 4 -1 2 1

Q3. [CO5, (Applying), 4+3= 7 marks] Write a C Program to perform the following tasks:

(a) You have to implement the game of word search. a grid (matrix) of characters of size $m \times n$ and a word is given, you have to determine if the word exists in the grid or not. The word can be constructed from letters of sequentially adjacent cells, where adjacent cells are horizontally or vertically neighboring. (User must enter size of grid, elements of grid and the word to be searched)



Example-Input: Enter grid size m,n : 5 5

Enter words in the matrix:

abcde

circl

derta

zebra

nerty

Enter the word to be searched: acdzn

Output: The word acdzn exists in the matrix.

(b) Create a new grid by applying the smoothing algorithm to the above grid. The smoothing algorithm should replace each character in the grid with the most frequently occurring character among its neighboring characters (up, down, left and right). If there is a tie in frequency, select the character that has least ASCII value.

Q4. [CO5, (Applying), 3+2= 5 marks] Give the output of the following program

```
(a) #include<stdio.h>
int main()
{
    int arr[2][2][3]={10,11,12,3,14,15,16,17,18,19,1,
21};
    char str[]="HelloWorld123";
    int (*ptr)[2][3]=arr+1;
    char *p=str+5;
    printf("%d",*(*(ptr)+1)+0));
    printf("%d",*(*(arr[0]+1)+2));
    printf("%s", p+*(*(arr[1]+1)+2)-19);
    printf("%d",*(*(ptr+0)+0))+*(p+2)-1);
    printf("%d",*(*(arr[0]+0)+(*(*(ptr+0)+1)+1)));
    printf("%c",*(p+*(*(ptr-1)+1)+0));
    return 0;
}
```

```
(b) #include<stdio.h>
int main()
{
    char str1[]="output analysis", str2[]="string
challenge", *p1=str1, *p2=str2, *start=str1, *end;

    for(int i=0; *p1!='\0' && *p2!='\0'; i++,p1++,p2++)
    {
        if(i%2==0)
        {
            char temp=*p1;
            *p1=*p2;
            *p2=temp;
        }
        for(end=str1; *end!='\0'; end++);
        for(end--; start<end; start++, end--)
        {
            char temp=*start;
            *start=*end; *end=temp;
        }
        printf("Final str1 :%s\n", str1);
        printf("Final str2: %s\n",str2);
        return 0;
    }
}
```

for (i=0; i<n; i++)

value = value + arr[i] (pow(x, i))